



TCP and OSPF Assignment ELC 3080

Name	ID
عبدالرحمن أحمد عبدالحميد العدوي	9202825
على أيمن محمد فولى	9202891

Submitted To: Prof. Dr. Khaled Fouad
3rd Year – Spring 2023

Contents

1. TCP Lab.....	3
1.1. Effect of TCP Window Size.....	3
1.1.1. Vary iperf3 window size from 1Kbytes to 6 Kbytes in increments of 1 Kbyte, then set it to 12, 16, 24, 32 Kbytes. Plot the average throughput and the average number of retransmissions as function of window size. Comment on the results and explain the zigzag behaviour noticed for larger window sizes. Note that number of retransmissions is the 5th column in iperf3 default output & throughput is the 4 th . You can plot data using matlab/python/any online graphing tool (Ex. https://chart-studio.plotly.com/create/#/). The window size is the last option in the iperf command (return to INTRO Lab).....	3
1.1.2. Click on any of the TCP a data segment whose source is node n7, dissect the segment by following different protocol layer headers from TCP->IP->Ethernet identifying how many header bytes are added by each layer, identify the TCP options used.....	6
1.1.3. Repeat for an ACK packet sent from node n11.....	7
1.2. TCP short VS long paths.....	7
1.2.1. Compare the result of throughput with the case when connection was made to node n11. Why throughput drops when connecting to n8 although capacities on the two paths are the same.....	8
1.3. Higher link Capacity with Drops VS Reliable Lower Capacity	8
1.3.1. Compare throughputs in cases a, b, c. Why b is better than c?.....	10
1.3.2. Compare throughputs in cases b, c and d. Which is better? Why?	10
1.3.3. Compare throughputs in cases e and f? Which is better? Why?	10
2. OSPF Lab.....	10
2.1. OSPF Link Cost Change	11
2.1.1. Check what happens to the path between n7 and n11 (as seen after steps 3 and 6)? Explain what happens. To help visualize the path choose toolbar->widgets-> Throughput.....	11
2.1.2. Set the cost of eth1 at node n5 back to 10. Establish two iperf3 connections: one from n7 to n11 and the second from n11 to n7 both for duration of 500 seconds. Now go to node n4 and set interface cost for interface connecting n4 with n5 to 40. What happens in the paths of the two connections? Explain what happens. What do you conclude?.....	12
2.2. OSPF Database Updates	12
2.2.1. Capture and explain the outputs due to execution of step 2. Why some destinations have more than route in the routing tables?.....	13
2.2.2. After executing step 3, determine how long it took the network to exchange link state packets and adjust routing tables. (Hint: you can calculate the required time by observing the time of first OSPF update message and the last ACK from Wireshark).....	14
2.2.3. After execution of step 4, identify the new routing table and router database at router n2. Explain the updates in the new routing table and the new database.....	14
2.3. OSPF Link State Advertisement Periodicity.....	15
2.3.1. Bring router n4 down and then up again and determine how long it took the network to recognize the router is down/up.	15
2.3.2. Now, the following is a bonus question (extra marks). Explain the noticed behavior of the link state update packet storms (flooding) where it generally takes longer times for the LSP exchange to die out (i.e. LSP exchange stops) when a router is down as compared with the case when the router is up again.....	16

1. TCP Lab

Each of the following sections describes an experiment based on the network topology provided by the [Project OSPF TCP.imn](#) file. In your report, divide the answers to correspond to one of these sections in exactly the same sequence and same name, for example use Question 2.1, Question 3.2 and so on. The [Introductory Lab](#) explained and introduced most of the needed tools like iperf3, vtysh, wireshark, etc. If you have not gone through it, do not start working on the assignment below. It is better to start the network emulation once and run Wireshark captures. You can reset Wireshark captures from experiment to another.

1.1. Effect of TCP Window Size

- Set wireshark filter to display TCP packets only.
- Start an iperf3 server on node n11.
- Start an iperf3 client on node n7 connecting to server on node n11 for a duration of 40 seconds and reporting interval of 10. **Note that in iperf, the client is the node sending the traffic and the server simply receives and sends an ACK.**

1.1.1. Vary iperf3 window size from 1Kbytes to 6 Kbytes in increments of 1 Kbyte, then set it to 12, 16, 24, 32 Kbytes. Plot the average throughput and the average number of retransmissions as function of window size. Comment on the results and explain the zigzag behaviour noticed for larger window sizes. Note that number of retransmissions is the 5th column in iperf3 default output & throughput is the 4th. You can plot data using matlab/python/any online graphing tool (Ex. <https://chart-studio.plotly.com/create/#/>). The window size is the last option in the iperf command (return to INTRO Lab).

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 1k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52948 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  4.03 MBytes  3.38 Mbits/sec  0    5.45 KBytes
[ 4] 10.00-20.00 sec  4.03 MBytes  3.38 Mbits/sec  0    5.45 KBytes
[ 4] 20.00-30.00 sec  4.03 MBytes  3.38 Mbits/sec  0    5.45 KBytes
[ 4] 30.00-40.00 sec  4.03 MBytes  3.38 Mbits/sec  0    5.45 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  16.1 MBytes  3.38 Mbits/sec  0      sender
[ 4] 0.00-40.00 sec  16.1 MBytes  3.38 Mbits/sec      receiver
```

Figure 1-1 1Kbyte windows size
Throughput: 0.4025 Mbyte/sec
Average number of retransmissions: 0

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 3k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52952 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  11.4 MBytes  9.52 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  45.4 MBytes  9.53 Mbits/sec  0      sender
[ 4] 0.00-40.00 sec  45.4 MBytes  9.53 Mbits/sec      receiver
```

Figure 1-1 3Kbyte windows size

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 2k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52950 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  9.08 MBytes  7.62 Mbits/sec  0    7.07 KBytes
[ 4] 10.00-20.00 sec  9.34 MBytes  7.84 Mbits/sec  0    7.07 KBytes
[ 4] 20.00-30.00 sec  9.38 MBytes  7.87 Mbits/sec  0    7.07 KBytes
[ 4] 30.00-40.00 sec  9.45 MBytes  7.93 Mbits/sec  0    7.07 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  37.3 MBytes  7.81 Mbits/sec  0      sender
[ 4] 0.00-40.00 sec  37.3 MBytes  7.81 Mbits/sec      receiver
```

Figure 1-1 2Kbyte windows size
Throughput: 0.9325 Mbyte/sec
Average number of retransmissions: 0

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52954 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  11.4 MBytes  9.54 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  45.4 MBytes  9.53 Mbits/sec  0      sender
[ 4] 0.00-40.00 sec  45.4 MBytes  9.53 Mbits/sec      receiver
```

Figure 1-1 4Kbyte windows size

Throughput: 1.135 Mbyte/sec
Average number of retransmissions: 0

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 5k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52956 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  11.4 MBytes  9.54 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  45.4 MBytes  9.53 Mbits/sec  0
[ 4] 0.00-40.00  sec  45.4 MBytes  9.53 Mbits/sec  0      sender
                                     receiver
```

Figure 1-1 5Kbyte windows size
Throughput: 1.135 Mbyte/sec
Average number of retransmissions: 0

Throughput: 1.135 Mbyte/sec
Average number of retransmissions: 0

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 6k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52958 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  154 KBytes  126 Kbits/sec  87    4.24 KBytes
[ 4] 10.00-20.00  sec  148 KBytes  122 Kbits/sec  90    4.24 KBytes
[ 4] 20.00-30.00  sec  148 KBytes  122 Kbits/sec  89    4.24 KBytes
[ 4] 30.00-40.00  sec  148 KBytes  122 Kbits/sec  90    4.24 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  600 KBytes  123 Kbits/sec  356
[ 4] 0.00-40.00  sec  588 KBytes  120 Kbits/sec  356      sender
                                     receiver
```

Figure 1-1 6Kbyte windows size
Throughput: 0.015 Mbyte/sec
Average number of retransmissions: 356

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 12k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52960 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  2.84 MBytes  2.38 Mbits/sec  481    4.24 KBytes
[ 4] 10.00-20.00  sec  3.32 MBytes  2.78 Mbits/sec  609    5.66 KBytes
[ 4] 20.00-30.00  sec  252 KBytes  206 Kbits/sec  110    4.24 KBytes
[ 4] 30.00-40.00  sec  252 KBytes  206 Kbits/sec  103    9.90 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  6.65 MBytes  1.39 Mbits/sec  1303
[ 4] 0.00-40.00  sec  6.63 MBytes  1.39 Mbits/sec  1303      sender
                                     receiver
```

Figure 1-1 12Kbyte windows size
Throughput: 0.16625 Mbyte/sec
Average number of retransmissions: 1303

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 16k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52962 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  431 KBytes  353 Kbits/sec  143    4.24 KBytes
[ 4] 10.00-20.00  sec  318 KBytes  261 Kbits/sec  122    4.24 KBytes
[ 4] 20.00-30.00  sec  382 KBytes  313 Kbits/sec  130    5.66 KBytes
[ 4] 30.00-40.00  sec  354 KBytes  290 Kbits/sec  128    4.24 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  1.45 MBytes  304 Kbits/sec  523
[ 4] 0.00-40.00  sec  1.43 MBytes  299 Kbits/sec  523      sender
                                     receiver
```

Figure 1-1 16Kbyte windows size
Throughput: 0.03625 Mbyte/sec
Average number of retransmissions: 523

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 24k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52964 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  329 KBytes  270 Kbits/sec  121    7.07 KBytes
[ 4] 10.00-20.00  sec  297 KBytes  243 Kbits/sec  120    7.07 KBytes
[ 4] 20.00-30.00  sec  297 KBytes  243 Kbits/sec  119    5.66 KBytes
[ 4] 30.00-40.00  sec  297 KBytes  243 Kbits/sec  120    4.24 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  1.19 MBytes  250 Kbits/sec  480
[ 4] 0.00-40.00  sec  1.15 MBytes  241 Kbits/sec  480      sender
                                     receiver
```

Figure 1-1 24Kbyte windows size
Throughput: 0.02975 Mbyte/sec
Average number of retransmissions: 480

```
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 32k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52966 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  274 KBytes  225 Kbits/sec  111    7.07 KBytes
[ 4] 10.00-20.00  sec  161 KBytes  132 Kbits/sec  95    19.8 KBytes
[ 4] 20.00-30.00  sec  214 KBytes  175 Kbits/sec  114    7.07 KBytes
[ 4] 30.00-40.00  sec  189 KBytes  155 Kbits/sec  98    15.6 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  839 KBytes  172 Kbits/sec  418
[ 4] 0.00-40.00  sec  789 KBytes  162 Kbits/sec  418      sender
                                     receiver
```

Figure 1-1 32Kbyte windows size
Throughput: 0.020975 Mbyte/sec
Average number of retransmissions: 418

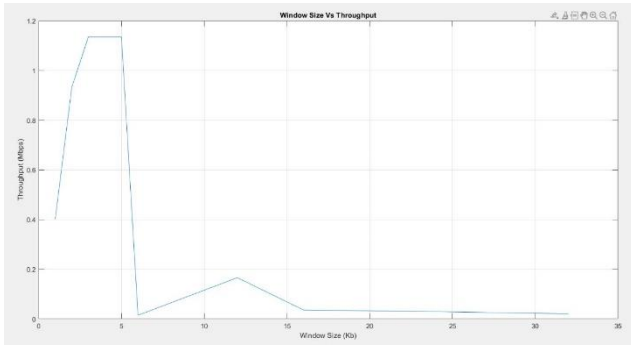


Figure 1-1 : throughput vs window size

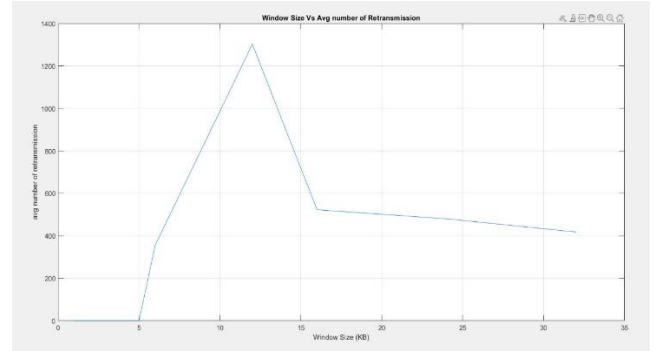


Figure 1-1 : Avg number vs window size

Comment on your findings:

Throughput = Data Transferred/Interval Time

we notice from figure of throughput vs window size that graph take zigzag behavior when we increase the window size from 1k to 5k and also the number of retransmissions is zero because the receiver buffer is not full yet so when we increase the window size, throughput also increase until receiver buffer is full and this happen when window size is 6k so we notice that throughput drop suddenly from graph and number of retransmissions increase because large number of packets lost and when this happen the transmitter realizes this problem and realizes to dedicate more space in the buffer so when window size is 12k the throughput increase again and number of retransmissions increase but when we increase window size more than 12k the problem of receiver buffer will happen again and the buffer is full because more packets will send to buffer and will be lost so the throughput will decrease again.

1.1.2. Click on any of the TCP a data segment whose source is node n7, dissect the segment by following different protocol layer headers from TCP->IP->Ethernet identifying how many header bytes are added by each layer, identify the TCP options used.

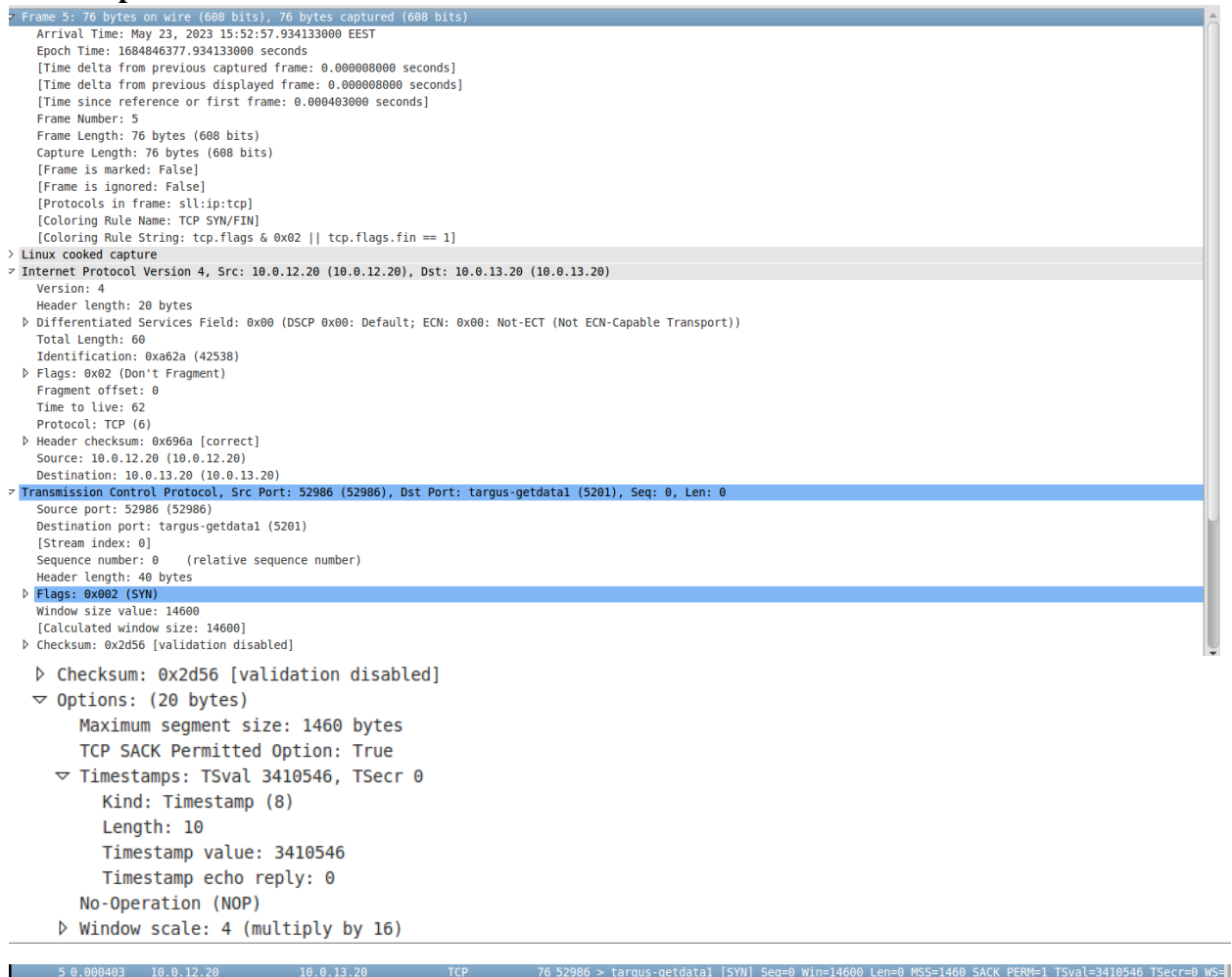


Figure 1-2 : TCP packet

Comment on your findings here:

- Frame : 76 bytes
- IP Header : 20 bytes
- TCP Header : 40 bytes
- TCP Options Timestamp : 20 bytes
- Ethernet Header : 16 bytes

1.1.3. Repeat for an ACK packet sent from node n11.

```
[Time delta from previous captured frame: 0.000220000 seconds]
[Time since reference or first frame: 0.000991000 seconds]
Frame Number: 10
Frame Length: 76 bytes (608 bits)
Capture Length: 76 bytes (608 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: sll:ip:tcp]
[Coloring Rule Name: TCP SYN/FIN]
[Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]
Linux cooked capture
Internet Protocol Version 4, Src: 10.0.13.20 (10.0.13.20), Dst: 10.0.12.20 (10.0.12.20)
  Version: 4
  Header length: 20 bytes
  Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not ECN-Capable Transport))
  Total Length: 60
  Identification: 0x0000 (0)
  Flags: 0x02 (Don't Fragment)
  Fragment offset: 0
  Time to live: 64
  Protocol: TCP (6)
  Header checksum: 0x0d95 [correct]
  Source: 10.0.13.20 (10.0.13.20)
  Destination: 10.0.12.20 (10.0.12.20)
Transmission Control Protocol, Src Port: targus-getdata1 (5201), Dst Port: 52986 (52986), Seq: 0, Ack: 1, Len: 0
  Source port: targus-getdata1 (5201)
  Destination port: 52986 (52986)
  [Stream index: 0]
  Sequence number: 0 (relative sequence number)
  Checksum: 0x2d56 [validation disabled]
  Options: (20 bytes)
    Maximum segment size: 1460 bytes
    TCP SACK Permitted Option: True
    Timestamps: TSval 3410547, TSecr 3410546
      Kind: Timestamp (8)
      Length: 10
      Timestamp value: 3410547
      Timestamp echo reply: 3410546
      No-Operation (NOP)
    Window scale: 4 (multiply by 16)
```

10.0.000991	10.0.13.20	10.0.12.20	TCP	76 targus-getdata1 > 52986 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=3410547
-------------	------------	------------	-----	--

Figure 1-3 ACK packet

Comment on Your findings:

- Frame : 76 bytes
- IP Header : 20 bytes
- TCP Header : 40 bytes
- TCP Options Timestamp : 20 bytes
- Ethernet Header : 16 bytes

1.2. TCP short VS long paths

- Run an iperf3 server on node n8.
- Start an iperf3 client on node n7 connecting to server on node n8 for a duration of 40 seconds and reporting interval of 10 with window size 4K.

1.2.1. Compare the result of throughput with the case when connection was made to node n11. Why throughput drops when connecting to n8 although capacities on the two paths are the same.

```

vcmd
root@n7:/tmp/pycore,51703/n7.conf# iperf3 -c 10.0.11.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.11.20, port 5201
[ 4] local 10.0.12.20 port 57973 connected to 10.0.11.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4]  0.00-10.00  sec  11.1 MBytes  9.27 Mbits/sec  0    14.1 KBytes
[ 4]  10.00-20.00  sec  11.0 MBytes  9.22 Mbits/sec  0    14.1 KBytes
[ 4]  20.00-30.00  sec  10.9 MBytes  9.13 Mbits/sec  0    14.1 KBytes
[ 4]  30.00-40.00  sec  11.0 MBytes  9.24 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4]  0.00-40.00  sec  43.9 MBytes  9.21 Mbits/sec  0
[ 4]  0.00-40.00  sec  43.9 MBytes  9.21 Mbits/sec
iperf Done.
root@n7:/tmp/pycore,51703/n7.conf#

```

Figure 1-4 : from n7 to n8

```

iperf Done.
root@n7:/tmp/pycore,51662/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 52954 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4]  0.00-10.00  sec  11.4 MBytes  9.54 Mbits/sec  0    14.1 KBytes
[ 4]  10.00-20.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4]  20.00-30.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
[ 4]  30.00-40.00  sec  11.4 MBytes  9.53 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4]  0.00-40.00  sec  45.4 MBytes  9.53 Mbits/sec  0
[ 4]  0.00-40.00  sec  45.4 MBytes  9.53 Mbits/sec
iperf Done.
root@n7:/tmp/pycore,51662/n7.conf#

```

Figure 1-4 : from n7 to n11

Comment in your findings here:

- Throughput from n7 as client to n8 as server: 1.0975 Mbyte /sec
 - Throughput from n7 as client to n11 as server: 1.135 Mbyte /sec
- we notice that when n8 become server instead of n11 the throughput decreases because the path from n11 to n7 is larger than path from n8 to n7 and so delay will increase

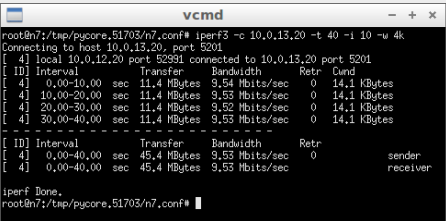
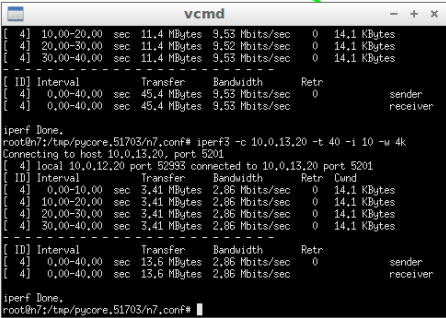
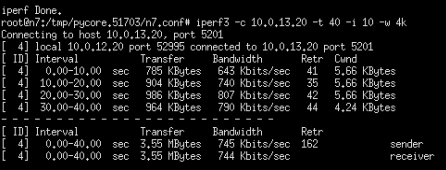
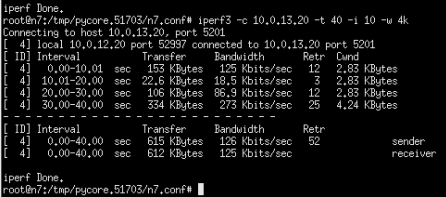
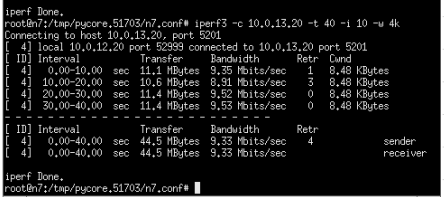
1.3. Higher link Capacity with Drops VS Reliable Lower Capacity

This questions for this part are based on the path between n7 and n11.

For each case of the following, run iperf3 client from n7 to n11 with window size 4K.

- Select link between n4 and n5, configure it to have capacity of 10 Mbps with zero loss in both directions.
- Select link between n4 and n5, configure it to have capacity of 3 Mbps with zero loss in both directions.
- Select link between n4 and n5, configure it to have capacity of 10 Mbps with 5% loss in both directions.
- Select link between n4 and n5, configure it to have capacity of 100 Mbps with 10% loss in both directions.
- Select link between n4 and n5, configure it to have capacity of 10 Mbps with 1% loss in direction from n4 to n5 and 0% loss in the other direction.
- Select link between n4 and n5, configure it to have capacity of 10 Mbps with 0% loss in direction from n4 to n5 and 1% loss in the other direction.

Please fill the following table with the AVG throughput of each case:

Case	Throughput	figures
A	1.135 Mbyte/sec	 <pre> vcmd root@n7:/tmp/pycore_51703/n7_conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 52991 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 11.4 MBytes 9.53 Mbits/sec 0 14.1 KBytes [4] 10.00-20.00 sec 11.4 MBytes 9.53 Mbits/sec 0 14.1 KBytes [4] 20.00-30.00 sec 11.4 MBytes 9.52 Mbits/sec 0 14.1 KBytes [4] 30.00-40.00 sec 11.4 MBytes 9.53 Mbits/sec 0 14.1 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 45.4 MBytes 9.53 Mbits/sec 0 [4] 0.00-40.00 sec 45.4 MBytes 9.53 Mbits/sec 0 sender receiver iperf Done. root@n7:/tmp/pycore_51703/n7_conf# </pre>
B	0.34 Mbyte/sec	 <pre> vcmd [4] 10.00-20.00 sec 11.4 MBytes 9.53 Mbits/sec 0 14.1 KBytes [4] 20.00-30.00 sec 11.4 MBytes 9.52 Mbits/sec 0 14.1 KBytes [4] 30.00-40.00 sec 11.4 MBytes 9.53 Mbits/sec 0 14.1 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 45.4 MBytes 9.53 Mbits/sec 0 [4] 0.00-40.00 sec 45.4 MBytes 9.53 Mbits/sec 0 sender receiver iperf Done. root@n7:/tmp/pycore_51703/n7_conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 52993 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 3.41 MBytes 2.86 Mbits/sec 0 14.1 KBytes [4] 10.00-20.00 sec 3.41 MBytes 2.86 Mbits/sec 0 14.1 KBytes [4] 20.00-30.00 sec 3.41 MBytes 2.86 Mbits/sec 0 14.1 KBytes [4] 30.00-40.00 sec 3.41 MBytes 2.86 Mbits/sec 0 14.1 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 13.6 MBytes 2.86 Mbits/sec 0 [4] 0.00-40.00 sec 13.6 MBytes 2.86 Mbits/sec 0 sender receiver iperf Done. root@n7:/tmp/pycore_51703/n7_conf# </pre>
C	0.08875 Mbyte/sec	 <pre> iperf Done. root@n7:/tmp/pycore_51703/n7_conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 52995 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 785 KBytes 643 Kbits/sec 41 5.66 KBytes [4] 10.00-20.00 sec 904 KBytes 740 Kbits/sec 35 5.66 KBytes [4] 20.00-30.00 sec 985 KBytes 807 Kbits/sec 42 5.66 KBytes [4] 30.00-40.00 sec 964 KBytes 790 Kbits/sec 44 4.24 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 3.55 MBytes 745 Kbits/sec 162 [4] 0.00-40.00 sec 3.55 MBytes 744 Kbits/sec 162 sender receiver </pre>
D	0.015375 Mbyte/sec	 <pre> iperf Done. root@n7:/tmp/pycore_51703/n7_conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 52997 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.01 sec 153 KBytes 125 Kbits/sec 12 2.83 KBytes [4] 10.01-20.00 sec 22.5 KBytes 18.5 Kbits/sec 3 2.83 KBytes [4] 20.00-30.00 sec 106 KBytes 86.9 Kbits/sec 12 2.83 KBytes [4] 30.00-40.00 sec 334 KBytes 273 Kbits/sec 25 4.24 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 615 KBytes 155 Kbits/sec 52 [4] 0.00-40.00 sec 612 KBytes 125 Kbits/sec sender receiver iperf Done. root@n7:/tmp/pycore_51703/n7_conf# </pre>
E	1.1125 Mbyte/sec	 <pre> iperf Done. root@n7:/tmp/pycore_51703/n7_conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 52999 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 11.1 MBytes 9.35 Mbits/sec 1 8.48 KBytes [4] 10.00-20.00 sec 10.5 MBytes 8.91 Mbits/sec 3 8.48 KBytes [4] 20.00-30.00 sec 11.4 MBytes 9.52 Mbits/sec 0 8.48 KBytes [4] 30.00-40.00 sec 11.4 MBytes 9.53 Mbits/sec 0 8.48 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 44.5 MBytes 9.33 Mbits/sec 4 [4] 0.00-40.00 sec 44.5 MBytes 9.33 Mbits/sec sender receiver iperf Done. root@n7:/tmp/pycore_51703/n7_conf# </pre>

F	0.4675 Mbyte/sec	1.3.1. Compare throughputs in cases a, b, c. Why b is better than c? Comment in your findings here:
---	------------------	---

we notice that throughput of case A is greater than throughput of case B and throughput of case B is greater than throughput of case C so case B is better than case C although capacity of the link in case of c is grater than case of b but losses in case c grater than case b so we need to retransmission packets in case c but in case b we do not need it because the losses is 0% and the throughput of case A is greater than case B because capacity of link in case of A is greater than case of b

1.3.2. Compare throughputs in cases b, c and d. Which is better? Why?
Comment in your findings here:

Case b better than case c and d because the loss in case b equal zero in both direction so client or server will not retransmit any packet although capacity in case b is lower than capacity in case c and d but in case c and d will retransmit packets because loss not equal zero

1.3.3. Compare throughputs in cases e and f? Which is better? Why?
Comment in your findings here:

case B better than case f because packets with large size is often send from n5 to n4 (from client to server) but the packets was sent from n4 to n5 is often with small size because it is just ACK or not ACK so losses will affect the link between n5 to n4 more than n4 to n5 because we need to retransmission large packet size

2. OSPF Lab

Each of the following sections describes an experiment based on the network topology provided by the [Project OSPF TCP.inn](#) file.

The [Introductory Lab](#) explained and introduced most of the needed tools like iperf3, vtysh, wireshark, etc. If you have not gone through it, do not start working on the assignment below.

It is better to start the network emulation once and run Wireshark captures. You can reset Wireshark captures from experiment to another.

2.1. OSPF Link Cost Change

1. Stop any running iperf3 clients.
2. Set all links to have zero loss in the two directions with 100 Mbps speed.
3. Run iperf3 between n7 and n11 for a duration of 500 seconds or longer. Identify the path between n7 and n11.
4. Open vtysh on node n5 by opening a bash terminal and typing vtysh. Now we can configure router and link costs.
5. Type the following in vtysh: show ip ospf interface eth1 (To know the ethernet number of each interface you can choose toolbar->view->show->interface names) This displays information about interface eth1. Note its ospf cost.
6. Type the following configure terminal
 - interface eth1 (the interface for the link between n5 and n4)
 - ospf cost 40 (set cost to 40)

2.1.1. Check what happens to the path between n7 and n11 (as seen after steps 3 and 6)? Explain what happens. To help visualize the path choose toolbar->widgets->Throughput.

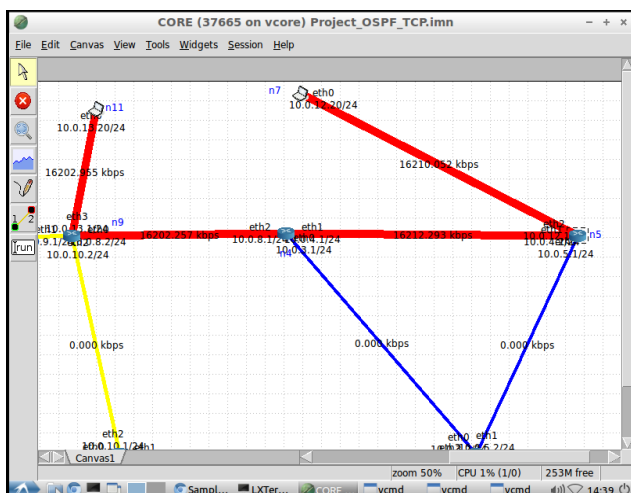


Figure 2-1 before

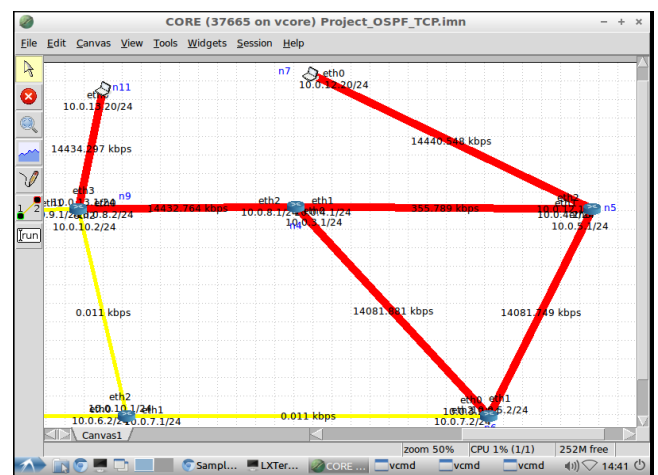


Figure 2-1 after increases the cost to 40

Comment on your findings:

When there is congestion due to high link cost, the router selects an alternate path with a lower cost using Dijkstra's Algorithm.

- 2.1.2. Set the cost of eth1 at node n5 back to 10. Establish two iperf3 connections: one from n7 to n11 and the second from n11 to n7 both for duration of 500 seconds. Now go to node n4 and set interface cost for interface connecting n4 with n5 to 40. What happens in the paths of the two connections? Explain what happens. What do you conclude?

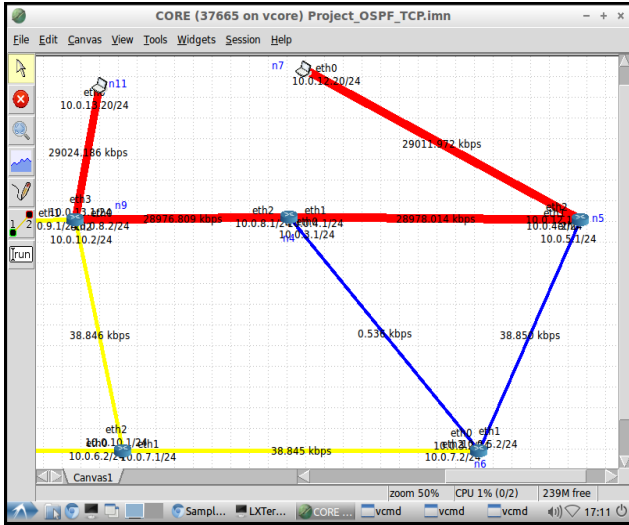


Figure 2-2 before

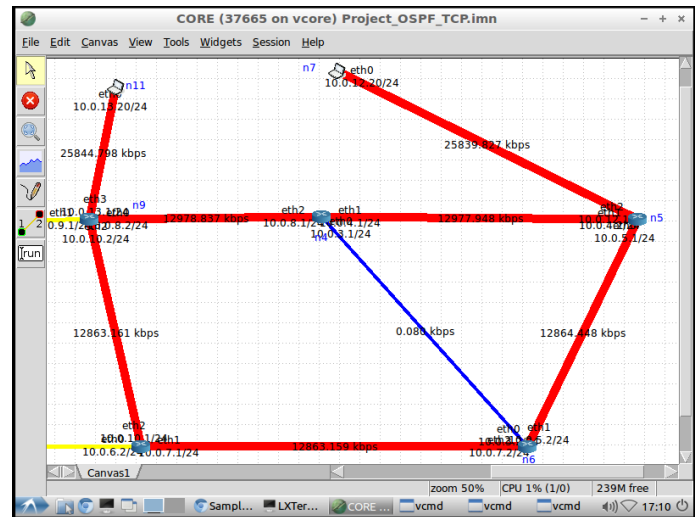


Figure 2-2 after increase the cost

Comment on your findings:

The path becomes much longer for the same reason mentioned above. The increasing cost of a link prompts the router to select another path, even if it is longer, to alleviate congestion.

2.2. OSPF Database Updates

1. Start wireshark and have its filter to capture only OSPF related packets.
2. Go to another router say router n2, open vtysh and issue the commands:
 - config terminal
 - show ip ospf database ([check this link to understand more](#))
 - show ip ospf route
3. On router n4, set link cost of eth1 to 20. Capture the link state packets advertised in Wireshark.
4. Go to router n4, go out of vtysh, or open new bash terminal and issue the following commands to disconnect router n4 from the network:
 - ifconfig eth0 down
 - ifconfig eth1 down
 - ifconfig eth2 down

2.2.1. Capture and explain the outputs due to execution of step 2. Why some destinations have more than route in the routing tables?

```

vcmd
N 10.0.13.0/24 via 10.0.1.1, eth0
via 10.0.9.1, eth2
[20] area: 0.0.0.0
via 10.0.9.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====

n2# show ip ospf database

OSPF Router with ID (10.0.1.2)

Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age Seq#      CkSum Link count
10.0.0.1     10.0.0.1     85 0x80000010 0xe6c3 3
10.0.0.2     10.0.0.2     76 0x80000010 0x8e0e 3
10.0.1.2     10.0.1.2     74 0x80000010 0x8110 3
10.0.3.1     10.0.3.1     1287 0x80000012 0x3732 3
10.0.3.2     10.0.3.2     67 0x80000011 0x9fe5 3
10.0.5.1     10.0.5.1     1037 0x8000000f 0x950c 3
10.0.6.2     10.0.6.2     85 0x8000000f 0xc0b1 3
10.0.8.2     10.0.8.2     25 0x80000011 0x181a 4

Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age Seq#      CkSum
10.0.0.2     10.0.0.2     425 0x80000007 0x53d1
10.0.1.2     10.0.1.2     1245 0x80000006 0x5ac7
10.0.2.1     10.0.1.2     454 0x80000007 0x45dc
10.0.3.2     10.0.3.2     1268 0x80000006 0x61b8
10.0.4.2     10.0.5.1     377 0x80000007 0x58bd
10.0.5.1     10.0.5.1     457 0x80000007 0x5db7
10.0.6.2     10.0.6.2     376 0x80000007 0x3fd2
10.0.7.1     10.0.6.2     415 0x80000007 0x4dc1
10.0.8.2     10.0.8.2     1186 0x80000006 0x48c2
10.0.9.1     10.0.8.2     405 0x80000007 0x21e9
10.0.10.2    10.0.8.2     1516 0x80000006 0x57ad

```

Figure 2-3 database

```

n2# show ip ospf route
===== OSPF network routing table =====
N 10.0.0.0/24 [20] area: 0.0.0.0
via 10.0.1.1, eth0
via 10.0.2.2, eth1
N 10.0.1.0/24 [10] area: 0.0.0.0
directly attached to eth0
N 10.0.2.0/24 [10] area: 0.0.0.0
directly attached to eth1
N 10.0.3.0/24 [30] area: 0.0.0.0
via 10.0.9.1, eth2
N 10.0.4.0/24 [50] area: 0.0.0.0
via 10.0.1.1, eth0
via 10.0.9.1, eth2
N 10.0.5.0/24 [40] area: 0.0.0.0
via 10.0.1.1, eth0
via 10.0.9.1, eth2
N 10.0.6.0/24 [20] area: 0.0.0.0
via 10.0.1.1, eth0
N 10.0.7.0/24 [30] area: 0.0.0.0
via 10.0.1.1, eth0
via 10.0.9.1, eth2
N 10.0.8.0/24 [20] area: 0.0.0.0
via 10.0.9.1, eth2
N 10.0.9.0/24 [10] area: 0.0.0.0
directly attached to eth2
N 10.0.10.0/24 [20] area: 0.0.0.0
via 10.0.9.1, eth2
N 10.0.11.0/24 [20] area: 0.0.0.0
via 10.0.2.2, eth1
N 10.0.12.0/24 [50] area: 0.0.0.0
via 10.0.1.1, eth0
via 10.0.9.1, eth2
N 10.0.13.0/24 [20] area: 0.0.0.0
via 10.0.9.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====

n2#

```

Figure 2-3 route

Comment on your findings:

Because there are more than path has the same cost. So, switch can select any one of them. Also, if one of the paths is down, there should be another one.

2.2.2. After executing step 3, determine how long it took the network to exchange link state packets and adjust routing tables. (Hint: you can calculate the required time by observing the time of first OSPF update message and the last ACK from Wireshark).

No.	Time	Source	Destination	Protocol	Length	Info
356	23.089026	fe80::200:ff:feaa:19	ff02::5	OSPF	92	Hello Packet
357	23.089037	fe80::200:ff:feaa:19	ff02::5	OSPF	92	Hello Packet
358	23.089101	fe80::200:ff:feaa:8	ff02::5	OSPF	96	Hello Packet
359	23.089225	fe80::200:ff:feaa:19	ff02::5	OSPF	92	Hello Packet
360	23.145243	10.0.0.1	224.0.0.5	OSPF	80	LS Acknowledge
361	23.145292	10.0.0.1	224.0.0.5	OSPF	80	LS Acknowledge
362	23.145356	10.0.2.2	224.0.0.5	OSPF	80	LS Acknowledge
363	23.145371	10.0.2.2	224.0.0.5	OSPF	80	LS Acknowledge
...	...	...	...	...	...	...
384	23.266721	10.0.8.2	224.0.0.5	OSPF	80	LS Acknowledge
385	23.266912	10.0.8.2	224.0.0.5	OSPF	80	LS Acknowledge
386	23.267063	10.0.8.2	224.0.0.5	OSPF	80	LS Acknowledge
387	23.286601	10.0.7.1	224.0.0.5	OSPF	80	LS Acknowledge
388	23.286661	10.0.7.1	224.0.0.5	OSPF	80	LS Acknowledge
389	23.286721	10.0.10.1	224.0.0.5	OSPF	80	LS Acknowledge
390	23.286736	10.0.10.1	224.0.0.5	OSPF	80	LS Acknowledge
391	23.286792	10.0.7.1	224.0.0.5	OSPF	80	LS Acknowledge
392	23.286912	10.0.10.1	224.0.0.5	OSPF	80	LS Acknowledge
397	31.530003	10.0.5.1	224.0.0.5	OSPF	84	Hello Packet

Figure 2-4 Put your screenshots here

Comment on your findings:

Time taken = 23.286912 – 23.145243 = 0.141669 sec

2.2.3. After execution of step 4, identify the new routing table and router database at router n2. Explain the updates in the new routing table and the new database.

```

root@n2:/tmp/pycore.60994/n2.conf# vtysh

Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

OSPF Router with ID (10.0.1.2)

  Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1     1768 0x8000000b 0xf0be 3
10.0.0.2     10.0.0.2     58  0x8000000d 0x940b 3
10.0.1.2     10.0.1.2     77  0x8000000c 0x890c 3
10.0.3.1     10.0.3.1     176 0x80000015 0xe7c8 2
10.0.3.2     10.0.3.2     140 0x8000000d 0xf0a8 3
10.0.5.1     10.0.5.1     131 0x8000000a 0xffa8 3
10.0.6.2     10.0.6.2     1759 0x8000000b 0xc8ad 3
10.0.8.2     10.0.8.2     127 0x8000000d 0x2823 4

  Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     688 0x80000003 0x5bcd
10.0.1.2     10.0.1.2     697 0x80000003 0x50c4
10.0.2.1     10.0.1.2     707 0x80000003 0x4dd8
10.0.5.1     10.0.5.1     690 0x80000003 0x65b3
10.0.6.2     10.0.6.2     1819 0x80000002 0x49cd
10.0.7.1     10.0.7.1     1349 0x80000002 0x57bc
10.0.9.1     10.0.8.2     1628 0x80000002 0x2be4
10.0.10.2    10.0.8.2     1818 0x80000002 0x5fa9

n2#
n2#

```

Figure 2-5 database

```

n2# show ip ospf route
===== OSPF network routing table =====
N   10.0.0.0/24
    [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.2.2, eth1
N   10.0.1.0/24
    [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24
    [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24
    [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.3.1, eth2
N   10.0.4.0/24
    [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.3.1, eth2
N   10.0.5.0/24
    [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.3.1, eth2
N   10.0.6.0/24
    [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.7.0/24
    [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.3.1, eth2
N   10.0.8.0/24
    [20] area: 0.0.0.0
    via 10.0.3.1, eth2
N   10.0.9.0/24
    [10] area: 0.0.0.0
    directly attached to eth2
N   10.0.10.0/24
    [20] area: 0.0.0.0
    via 10.0.3.1, eth2
N   10.0.11.0/24
    [20] area: 0.0.0.0
    via 10.0.2.2, eth1
N   10.0.12.0/24
    [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.3.1, eth2
N   10.0.13.0/24
    [20] area: 0.0.0.0
    via 10.0.3.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====

n2#

```

Figure 2-5 route

Comment on your findings:

When we disconnect the N4 node, there should be another node to take its place, which is N2. N2 has a higher cost than N4, so the update will be shown in the database.

2.3. OSPF Link State Advertisement Periodicity

This part is on your own. No instructions are given. Bring router n4 ethernet interfaces back up.

Stop any previous iperf3 connections. Let Wireshark capture focus on OspfV2 only (IPv4).

Apply the filter (ospf.msg.lsupdate || ospf.msg.lsack) && ip.addr == 224.0.0.5

2.3.1. Bring router n4 down and then up again and determine how long it took the network to recognize the router is down/up.

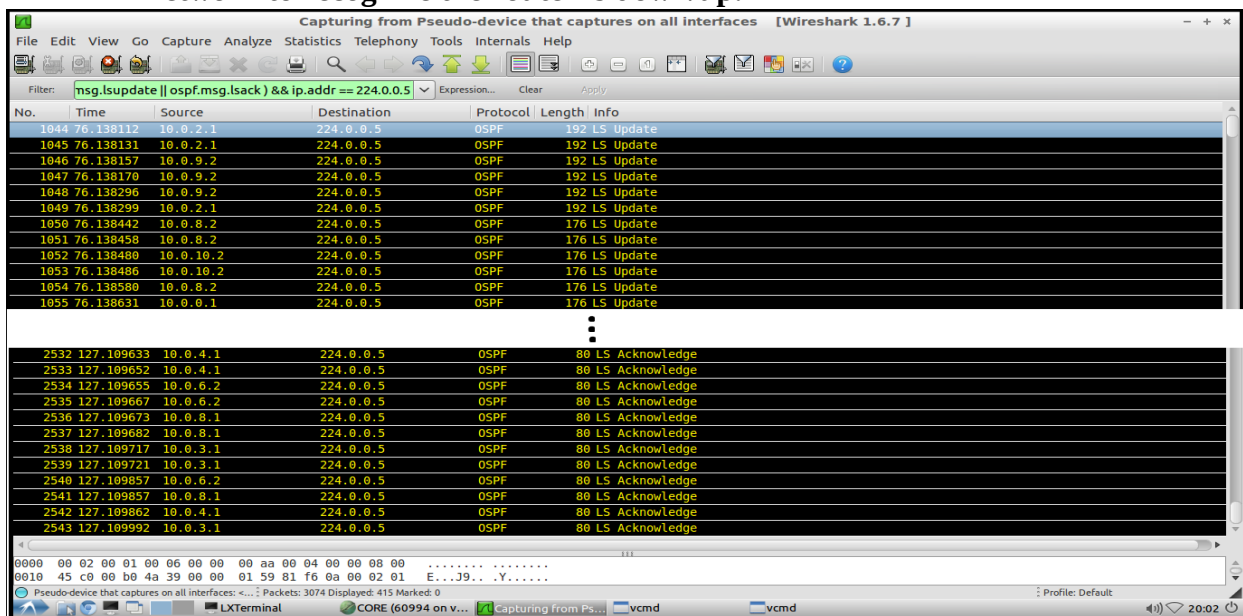


Figure 2-6 down time

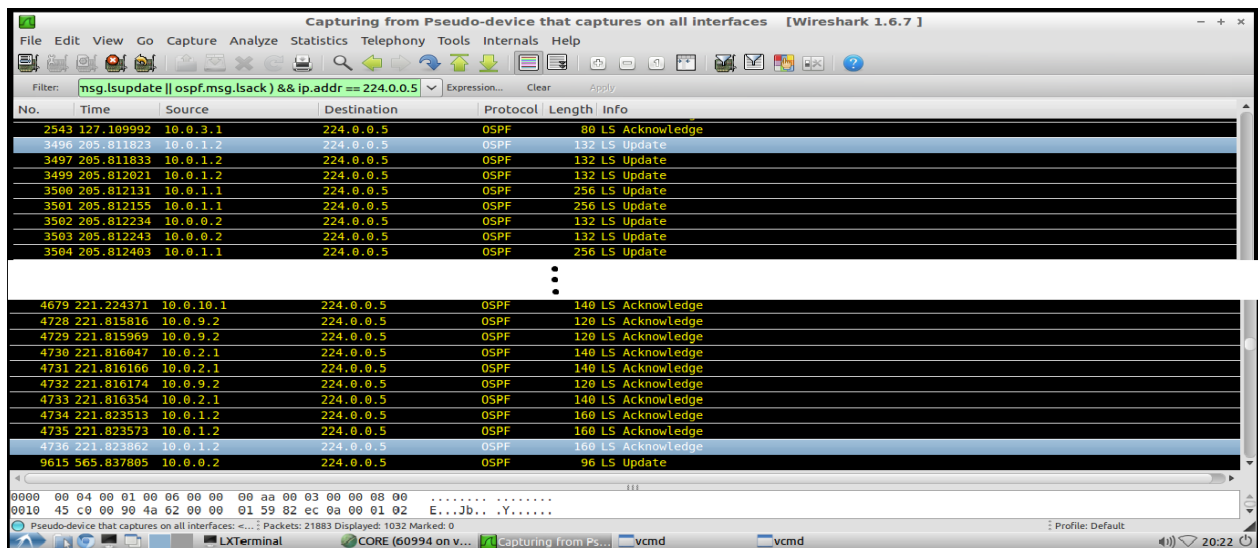


Figure 2-6 Up time

Comment on your findings:

Time taken to recognize it's DOWN = $127.109992 - 76.138112 = 50.97188$ sec

Time taken to recognize it's UP = $221.823862 - 205.811823 = 16.012039$ sec

- 2.3.2. Now, the following is a bonus question (extra marks). Explain the noticed behavior of the link state update packet storms (flooding) where it generally takes longer times for the LSP exchange to die out (i.e. LSP exchange stops) when a router is down as compared with the case when the router is up again.**

Comment :

When a router goes down, it may take some time for the other routers in the network to recognize that it is no longer available, When a router goes down, it stops sending hello packets or responding to them. The other routers in the network will continue to send hello packets to the failed router, but will not receive any response. After a certain amount of time (known as the "dead interval"), the other routers will declare the failed router as down and will update their routing tables accordingly. As a result, the other routers may continue to treat the router as down for a certain amount of time (known as the "hold time"), even though it is actually up again.